OpenMontage 从入门到精通

> **版本依据**：calesthio/OpenMontage 官方仓库（main 分支）`README.md`、`README_zh-CN.md` 及官方网站 openmontage.video；核实日期为 2026 年 10 月 22 日。 **许可证**：GNU AGPLv3。该许可证对以网络服务形式提供本系统的衍生部署有源码公开要求，商用前应阅读仓库内 `LICENSE` 了解相关义务。 **读者假设**：具备命令行基础，已在使用或准备使用某款 AI 编程助手（Claude Code、Cursor、GitHub Copilot、Windsurf、Codex 等）。 **平台覆盖**：安装与命令同时给出 macOS / Linux 与 Windows PowerShell 两种版本。 **成本口径**：文中美元金额均为官方示例值或官方标注的量级区间，实际支出以系统在提案阶段给出的成本估算为准；各提供商的定价与免费额度以仓库内 `docs/PROVIDERS.md` 为准。 **口径说明**：官方中文 README 的规模数字（52 个工具、400+ 项技能、15 家视频提供商）较英文版滞后，本教程统一采用英文 README 的口径：12 条生产流水线、100+ 注册生产工具、60+ 提供商集成、700+ 智能体技能与生产知识文件、20+ JSON Schema。 **内容边界**：商业版 OpenMontage Studio 尚处私测阶段，本教程仅依据官网公开信息简述，不涉及其未公开的内部功能。

OpenMontage 是一套由 AI 编程助手驱动的开源视频制作系统：使用者用自然语言提出制作需求，智能体（Agent）按流水线完成研究、脚本、分镜、素材、配音、合成与质检。本教程按「入门篇、进阶篇、高级篇、精通篇」四部分组织：入门篇完成安装并产出第一个视频；进阶篇掌握流水线、提供商与创意控制；高级篇深入渲染引擎、本地 GPU、风格系统与治理机制；精通篇面向架构理解、定制开发与综合实战。各章可独立查阅，但章节顺序即推荐学习路径。

第一部分 入门篇

第 1 章 认识 OpenMontage

本章解决三个问题：OpenMontage 的定位是什么，它与常见在线 AI 视频工具的本质区别在哪里，使用前必须先理解哪些核心概念。完成本章学习后，读者应能够判断该系统是否适合当前的制作需求，并在后续章节中准确理解各个组成部分的名称与作用。

1.1 项目定位

OpenMontage 的官方表述为「首个开源的智能体（agentic）视频制作系统」。项目仓库位于 GitHub（calesthio/OpenMontage），采用 GNU AGPLv3 许可证，官方吉祥物为打板器形象 Monty。项目的口号是「将 AI 编程助手变成一个完整的视频制作工作室」。

其运行形态与常见的在线视频生成器不同：OpenMontage 是一套安装在本地的代码仓库，本身不包含视频生成的模型推理服务，也不提供网页界面作为主要入口。它的编排核心是 AI 编程助手——Claude Code、Cursor、GitHub Copilot、Windsurf、Codex 等任何能够读取仓库文件并执行 Python 的工具。使用者向助手描述制作需求，助手作为智能体按照仓库内定义的流水线执行各阶段工作，期间调用仓库内注册的 Python 工具完成具体的生成与处理任务。

另外应当区分两个名称相近的事物：本教程介绍的开源仓库是本体的核心；英文官方网站 openmontage.video 上展示的 OpenMontage Studio 是同团队开发的商业产品，整合了对话式工作台与插件机制（支持在 Claude Code、Codex 中以插件或 MCP 形式调用创意工具），截至核实日期处于私测阶段。二者的流水线理念一致，但本教程全部内容以开源仓库为准。

1.2 与在线 AI 视频工具的区别

多数「一句话生成一个视频片段」的在线工具与 OpenMontage 属于两类产品。下表列出主要差异。

对比维度	常见在线生成工具	OpenMontage
输入方式	一条提示词	一段自然语言需求描述，可附参考视频
产出单位	单个片段（clip）	一部完整成品：研究、脚本、旁白、画面、音乐、字幕、时间线
制作过程	黑盒，一次调用返回结果	分阶段推进，每阶段产出可读文档与资产，关键节点等待人工批准
事实依据	通常不含联网检索	脚本写作前先进行 15–25 次以上的网络检索（YouTube、Reddit、新闻与学术站点等），产出带引用的研究简报
素材来源	主要为 AI 生成	图像动画、真实素材（免费档案与图库）、AI 生成视频三条路径均可
成本	按订阅或按次计费，账单在订阅层面	每条提示词、每项动作均先估算后执行，单次审批阈值与总预算上限可配置
质量控制	无系统性质检	合成前验证、渲染后自我审查、PPT（幻灯片）风险评分等多重关卡
依赖锁定	服务商绑定	提供商可自由替换，本地工具与云端 API 并存，无供应商锁定

其中「真实素材路径」是官方特别强调的区分点：系统可以从 Archive.org、NASA、Wikimedia Commons 等免费开放档案与图库建立可检索语料库，检索真实动态影像，剪辑成时间线并渲染成品，而不只是「让几张静态图片动起来」。

1.3 核心概念速览

后续章节会反复出现以下术语。此表给出速览定义，附录 C 提供更完整的术语表。

术语	英文	含义
智能体	Agent	指正在运行 OpenMontage 的 AI 编程助手；它既是编排器，也是执行者
流水线	Pipeline	从创意到成品的完整生产工作流，共 12 条，按内容类型划分
阶段	Stage	流水线中的一环，如研究、提案、脚本等，全流程共七个阶段
导演技能	Director Skill	教导智能体如何执行某个阶段的 Markdown 指令文件
流水线清单	Pipeline Manifest	位于 pipeline_defs/ 的 YAML 文件，定义阶段、工具、审查标准与成功关卡
工具注册表	Tool Registry	自动发现并注册所有 Python 工具的机制，用于查询当前可用的能力与提供商
提供商	Provider	具体执行生成的服务或模型，如 Kling、FLUX、ElevenLabs、Piper 等
质量检验门	Quality Gate	系统自动执行的质检机制，如合成前验证、渲染后自我审查
审批门	Approval Gate	需要人工批准才能继续的创意决策点
检查点	Checkpoint	各阶段保存的状态 JSON，含决策日志与成本快照，支持中断后恢复
决策日志	Decision Log	记录每一次重大选择及其备选项、置信度与理由的审计记录
Backlot 看板	Backlot	本地运行的生产状态看板，实时展示各阶段进展与资产

1.4 系统的产出路径

OpenMontage 能够制作的视频分为四类路径，理解它们有助于在第 6 章正确书写提示词。

第一类是**图像动画视频**：智能体先生成或获取静态图像，再由 Remotion 渲染引擎为图像加入弹簧动画、镜头运动、粒子叠加与转场，最终形成带旁白、音乐与词级字幕的成品。未配置任何视频生成提供商时默认走此路径，官方多个示例（成本约 0.15 美元的吉卜力风格短片）即属此类。

第二类是**本地角色动画**：使用 SVG 绑定、姿势库与 GSAP 时间线制作卡通角色表演，由 HyperFrames 引擎渲染，输出至 projects/<项目名>/renders/final.mp4。

第三类是**真实素材视频**：由纪录片蒙太奇流水线从免费开放来源（Archive.org、NASA、Wikimedia Commons，以及可选的 Pexels、Unsplash）建立支持 CLIP 语义检索的语料库，剪辑真实动态影像。此路径不需要付费生成 API。

第四类是**AI 生成视频**：配置视频生成提供商（如 Kling、Runway、Veo 等）后，智能体可调用云端模型生成动态片段，再与其他资产合成。此路径成本最高，也是官方示例「THE LAST BANANA」（总成本 1.33 美元，6 个 Kling v3 片段）所走的路径。

1.5 规模与官方示例

截至核实日期，官方英文 README 标注的规模为：12 条生产流水线、100 多个注册生产工具、60 多个提供商集成、700 多个智能体技能与生产知识文件、20 多个 JSON Schema。官方频道（YouTube @OpenMontage）发布的每条示例视频均附完整提示词、所用流水线、工具与成本，可据此复现。

部分官方示例影片的成本数据如下表所示，可作为成本预期参考。

示例影片	形式	关键配置	官方标注成本
Afternoon in Candyland	吉卜力风格短片	12 张 FLUX 图像、粒子叠加、环境音乐、Remotion 合成	0.15 美元
VOID — Neural Interface	产品广告	仅 OpenAI 一个密钥：GPT 图像、TTS 旁白、WhisperX 词级字幕、Remotion 数据可视化	0.69 美元
THE LAST BANANA	60 秒动画短片	6 个 Kling v3 片段（经 fal.ai）、Chirp3-HD 旁白、免版税钢琴曲、Remotion 合成	1.33 美元
Reimagine Your Universe	50 秒竖屏影片	5 个生成动态场景、Google Chirp 旁白、Pixabay 配乐、HyperFrames 合成	约 4 美元

第 2 章 环境准备与安装

本章解决「从零到可验证安装完成」的问题。完成本章学习后，读者应能在 macOS、Linux 或 Windows 上完成安装，并通过工具注册表确认系统真实可用能力。需要说明的是，OpenMontage 安装成功的判断标准不只是依赖安装完毕，而是「注册表能列出能力与提供商、后续能产出媒体文件」，本章的验证命令即服务于该标准。

2.1 系统要求

组件	版本要求	获取方式
Python	3.10 及以上	python.org
Node.js	18 及以上（使用 HyperFrames 引擎时需 22 及以上）	nodejs.org
FFmpeg	无特定版本要求，但必须可在命令行调用	brew、apt 或 ffmpeg.org
Git	用于克隆仓库与版本管理	git-scm.com
AI 编程助手	Claude Code、Cursor、Copilot、Windsurf 或 Codex 等任意可读文件、可执行 Python 的助手	各官方渠道

Node.js 的两档版本要求来源于两类渲染引擎：基础安装与 Remotion 引擎要求 Node.js 18 及以上，HyperFrames 引擎要求 Node.js 22 及以上，详见第 8 章。

2.2 安装前自检

不要等到安装中途失败后才检查环境。在终端执行以下命令逐一确认版本，任何一条无法通过都应先补齐对应组件。

```
python3 --version
node --version
npm --version
ffmpeg -version
git --version
```

判读标准：Python 输出不低于 3.10，Node.js 输出不低于 18（或 22），FFmpeg 命令能够输出版本信息。FFmpeg 是后期编码、字幕烧录与音频混音的基础，若此命令不可用，后续合成阶段不可能完整通过。

macOS 与 Linux 安装 FFmpeg 的官方命令如下。

```
# macOS (Homebrew)
brew install ffmpeg

# Ubuntu/Debian
sudo apt update
sudo apt install ffmpeg
```

Windows 没有官方包管理器命令，建议从 ffmpeg.org 下载构建包，解压后将其 bin 目录加入 PATH 环境变量，随后重新打开终端验证。

2.3 标准安装

macOS 与 Linux 的最短路径为三行命令。

```
git clone https://github.com/calesthio/OpenMontage.git
cd OpenMontage
make setup
```

`make setup` 会创建 Python 虚拟环境、安装 Python 依赖、进入 remotion-composer 目录安装 Node 依赖、安装本地语音合成工具 Piper，并从模板生成 .env 配置文件。

克隆仓库需要能够访问 GitHub；网络受限环境下可以使用镜像托管平台上的同步仓库作为获取渠道，但后续更新仍应以官方仓库为来源，避免引入未经官方确认的改动。

2.4 无 make 时手动安装（macOS/Linux）

没有 make 命令的环境中，按官方给出的单行命令手动执行，使用虚拟环境避免把依赖写入系统 Python。

```
python3 -m venv .venv && source .venv/bin/activate && python -m pip install -r requirements.txt && cd remotion-composer && npm install && cd .. && python -m pip install piper-tts && cp .env.example .env
```

其中 `python3 -m venv .venv` 创建名为 .venv 的虚拟环境；`source .venv/bin/activate` 激活该环境；`pip install -r requirements.txt` 安装 Python 依赖；`npm install` 在 remotion-composer 子目录安装 Remotion 合成引擎的 Node 依赖；`pip install piper-tts` 安装免费的本地语音合成组件；`cp .env.example .env` 从模板复制环境配置文件。注意：重新打开终端时需要重新执行 `source .venv/bin/activate`。

2.5 Windows PowerShell 安装

Windows 下不要照抄 Bash 的 `source` 与 `cp` 命令，官方给出的 PowerShell 版本如下。

```
git clone https://github.com/calesthio/OpenMontage.git
Set-Location OpenMontage
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Set-Location remotion-composer
npm install
Set-Location ..
python -m pip install piper-tts
Copy-Item .env.example .env
```

两处已知的官方说明：

- 若 `npm install` 报错 `ERR_INVALID_ARG_TYPE`，官方建议改用 `npx --yes npm install`。
- 若 PowerShell 拒绝执行 Activate.ps1（提示执行策略限制），应确认执行策略设置，而不是反复重建环境。也可以不激活虚拟环境，直接以 `.\.venv\Scripts\python.exe` 运行后续所有 Python 命令。

2.6 安装验证：查询工具注册表

文件安装完成不代表所有视频工具都已被发现。在仓库根目录、虚拟环境已激活的状态下运行以下两条命令，分别查询当前机器的能力范围（support envelope）与可选提供商菜单。

```
python -c "from tools.tool_registry import registry; import json; registry.discover();
print(json.dumps(registry.support_envelope(), indent=2))"
```

```
python -c "from tools.tool_registry import registry; import json; registry.discover();
print(json.dumps(registry.provider_menu(), indent=2))"
```

两条命令均应正常退出并输出 JSON 结果。判读标准：命令非零退出、Python 导入失败、输出为空或缺少预期能力时，不应继续让智能体制作完整视频，而应先回到第 2.2 节排查环境。

2.7 将仓库交给 AI 编程助手

在 AI 编程助手中打开本仓库目录即可开始使用。仓库为各平台内置了专用指令文件，指向共享的 AGENT_GUIDE.md（操作指南与智能体契约）与 PROJECT_CONTEXT.md（架构参考）。

平台	配置文件
Claude Code	CLAUDE.md
Cursor	CURSOR.md 及 .cursor/rules/
GitHub Copilot	COPILOT.md 及 .github/copilot-instructions.md
Codex	CODEX.md
Windsurf	.windsurfrules

仓库更新属常规 Git 操作，在仓库目录执行 `git pull` 即可拉取官方最新提交；更新后建议重新运行第 2.6 节的验证命令确认能力无回归。

第 3 章 第一次出片：零 Key 路径

本章解决「不配置任何付费密钥就产出第一个视频」的问题。完成本章学习后，读者应能运行官方演示、发起第一个自制短片、定位输出文件并完成成果验收。零 Key 路径的成本为零，是验证安装、理解流水线行为的最稳妥起点。

3.1 零 Key 能力清单

`make setup` 完成后，无需任何付费 API 密钥即可使用以下能力。

能力	免费工具	说明
旁白配音	Piper TTS	免费离线的文本转语音，生成真人质感的旁白
开放影像素材	Archive.org + NASA + Wikimedia Commons	免费/开放档案影像、教育媒体与纪录片素材
额外图库素材	Pexels + Unsplash + Pixabay	免费图库影像与图像（开发者密钥可免费申请，见第 4 章）
React 合成	Remotion	弹簧动画图像场景、文字卡片、数据卡片、图表、词级字幕
HTML/GSAP 合成	HyperFrames	动态排版、产品宣传、发布短片、SVG 角色动画
后期制作	FFmpeg	编码、字幕烧录、音频混合、色彩调色
字幕生成	内置	带词级时间轴的自动字幕

所谓词级字幕，指字幕按单词（或中文分词）级时间戳逐词出现，而非整句一次性显示。

3.2 运行官方演示

仓库提供零 Key 演示命令，直接渲染示例视频。

```
make demo
```

该命令用于验证「从安装到渲染」链路是否畅通，不需要配置任何密钥。

3.3 发起第一个自制视频

在 AI 编程助手中打开仓库后，用一段明确的自然语言描述发起制作。官方给出的零 Key 示例如下（引号内为可直接复制的原文）。

```
"Make a 45-second animated explainer about why the sky is blue"
```

中文含义：制作一段 45 秒的动画解说，解释天空为什么是蓝色的。

```
"Create a 60-second video about the history of the internet, with narration and captions"
```

中文含义：制作一段 60 秒的关于互联网历史的视频，包含旁白与字幕。

```
"Make a data-driven explainer about coffee consumption around the world"
```

中文含义：制作一段关于全球咖啡消费的数据驱动型解说视频。

提示词中明确「时长、内容主题、是否需要旁白与字幕」三要素，有助于智能体在提案阶段给出准确的方案与成本估算。

3.4 选择免费路径

零 Key 状态下有两条近乎免费的制作路径，分别对应不同提示词写法。

图像动画路径：默认路径。Piper 负责旁白，图像（免费生成或图库获取）提供画面，Remotion 将其动画化成片。直接按第 3.3 节示例书写即可。

真实素材路径：由纪录片蒙太奇流水线剪辑真实影像。若想在提示词中触发该路径，官方要求明确使用以下关键词组合：提及「documentary montage（纪录片蒙太奇）」「tone poem（音画诗）」或「stock-footage collage（图库素材拼贴）」，并明确写出「use real footage only（只使用真实素材）」。官方示例如下。

```
"Make a 90-second documentary montage about what a city feels like at 4am. Use real footage only, no narration, elegiac tone."
```

中文含义：制作一段 90 秒的纪录片蒙太奇，表现凌晨四点城市的感觉；只使用真实素材，无旁白，挽歌式基调。

3.5 产出位置与成果验收

流水线会将项目文件与成品写在项目目录中。已知的成品路径为 projects/<项目名>/renders/final.mp4。定位新生成文件可执行：

```
find . -type f -name "*.mp4" -print
```

随后用 ffprobe（FFmpeg 自带的媒体信息探测工具）检查媒体流完整性：

```
ffprobe -v error -show_entries format=duration:stream=codec_type,codec_name -of json 输出文件.mp4
```

合格标准为：JSON 输出中包含非零时长的视频流；制作要求含旁白或音乐时，还应存在音频流。只有项目文件而没有最终 MP4，说明流水线未完成，不能采信智能体「已完成」的总结。

3.6 首次出片的范围控制

首次运行不建议直接制作长视频或复杂项目。合理的第一步是 15 至 30 秒的测试短片：时长短意味着调用次数少、排错周期短、即使失败也容易定位阶段。待首个短片通过第 3.5 节验收后，再逐步增加时长、素材类型与提供商。

第二部分 进阶篇

第 4 章 提供商与 API 密钥配置

本章解决「按需扩展系统能力」的问题。OpenMontage 的能力由已配置的密钥决定：每个密钥都是可选的，添加哪些就解锁哪些工具。完成本章学习后，读者应能按分类正确配置 .env 文件，理解各提供商的定位，并采取逐步启用、逐项验证的配置策略。

4.1 .env 文件机制

系统通过仓库根目录的 .env 文件读取密钥。安装步骤已从模板生成该文件：`cp .env.example .env`（Windows 为 `Copy-Item .env.example .env`）。向该文件添加「环境变量名=密钥值」格式的条目并保存后，重新运行第 2.6 节的注册表查询命令，即可看到新增的提供商能力。

环境变量（environment variable）是操作系统或程序读取的具名配置项；.env 文件是项目级集中管理这些配置的约定文件。要注意 .env 属于敏感信息，不应当提交到 Git 仓库，详见第 4.9 节。

4.2 密钥分类总表

官方 README 按用途列出以下密钥条目，均为可选配置。

环境变量	类别	解锁能力
FAL_KEY	图像 + 视频网关	FLUX 图像生成；Google Veo、Kling、MiniMax 视频；Recraft 图像
ATLASCLOUD_API_KEY	图像 + 视频网关	Seedream、Nano Banana、GPT Image 图像；Kling、Seedance、Hailuo 视频
KLING_API_KEY	视频官方直连	官方 Kling 视频、图像、TTS、数字人、唇形同步
KLING_API_BASE_URL	直连配置	可选；默认为新加坡 API 端点
PEXELS_API_KEY	免费素材库	免费图库视频与图像
PIXABAY_API_KEY	免费素材库	免费图库视频与图像
UNSPLASH_ACCESS_KEY	免费素材库	免费图库图像
SUNO_API_KEY	音乐	生成完整歌曲、伴奏，任意流派
ELEVENLABS_API_KEY	语音与图像	顶级 TTS、AI 音乐、音效
OPENAI_API_KEY	语音与图像	OpenAI TTS、GPT Image 图像
XAI_API_KEY	语音与图像	Grok 图像编辑/生成 + Grok 视频生成
GOOGLE_API_KEY	语音与图像	Google Imagen 图像、Google TTS（700+ 声音）
ARK_API_KEY	视频官方直连	火山方舟直连——Seedance 2.0 Standard/Fast/Mini
HEYGEN_API_KEY	视频网关	HeyGen 多模型网关（VEO、Sora、Runway、Kling）
RUNWAY_API_KEY	视频官方直连	Runway Gen-4 直连

4.3 视频生成提供商

视频生成是成本最高的一类能力。英文 README 标注为 20 多家提供商，主流条目如下表，完整列表以 docs/PROVIDERS.md 与注册表输出为准。

提供商	类型	说明
Kling (fal.ai)	云端 API	高质量、速度快，经 fal.ai 网关调用
Kling Official	云端 API	官方直连，独立提供商
Atlas Cloud	云端 API	统一网关，聚合 Seedance、MiniMax、混元等多家模型
Seedance 2.0（火山方舟直连）	云端 API	官方直连接口，独立提供商 seedance_ark
Seedance 2.5 / 2.0	云端 API	支持 Text、Image、Reference 三类视频生成工作流
Gemini Omni Flash	云端 API	对话式多模态视频生成与编辑
Runway Gen-4	云端 API	电影级质量，含 Gen-3 Alpha Turbo、Gen-4 Turbo、Gen-4 Aleph
Google Veo 3.1	云端 API	高端电影级视频，经 Google GenAI 或 fal.ai 接入
Grok Imagine Video	云端 API	基于参考图的视频与 xAI 原生短视频生成
Higgsfield	云端 API	多模型编排器，Soul ID 用于角色一致性
MiniMax / H3	云端 API	成本效益型，含文本、图像与参考驱动工作流
HeyGen	云端 API	多模型网关
WAN 2.1 / 2.2	本地 GPU	免费本地模型，含加速的 ComfyUI 工作流
混元 Hunyuan	本地 GPU	免费，质量较高
CogVideo	本地 GPU	免费，含 2B、5B 规格
LTX-Video	本地 GPU / Modal	本地免费，或以 Modal 自托管云端
Pexels、Pixabay、Wikimedia Commons	素材库	免费库存与档案视频

4.4 图像生成提供商

图像生成承担画面主视觉与视频首帧等职责，官方标注为 15 家以上，主流条目如下。

提供商	类型	说明
FLUX	云端 API	业界一流质量
Google Imagen	云端 API	Imagen 4，高质量、多种长宽比
Grok Imagine Image	云端 API	图像编辑、风格转换、多图合成能力较强
GPT Image 2	云端 API	OpenAI 图像模型
Seedream 5.0	云端 API	高保真文生图与图像编辑，经支持的网关接入
Nano Banana 2	云端 API	多模态图像生成与编辑
Atlas Cloud	云端 API	统一接入多个图像模型家族
Recraft	云端 API	面向设计的生成
Kling Official	云端 API	官方直连图像生成与参考工作流
Local Diffusion	本地 GPU	Stable Diffusion，免费
Pexels、Pixabay、Unsplash	素材库	免费图库图像
ManimCE	本地	数学与科学动画

4.5 语音、音乐与音效提供商

提供商	类型	说明
ElevenLabs	云端 API	语音质量较高，含 AI 音乐与音效
Google TTS	云端 API	700+ 声音、50+ 语言，适合本地化配音
Kling Official TTS	云端 API	官方 Kling 旁白，需要已知 voice_id
OpenAI TTS	云端 API	速度快、价格适中
Piper	本地	完全免费、可离线
Azure Speech	云端 API	多语言语音服务
DashScope / 豆包 / Fish Audio	云端 API	多语言与表现力语音选项
Suno AI	云端 API	生成含人声与歌词的完整歌曲，任意流派，最长 8 分钟
ElevenLabs Music	云端 API	AI 音乐生成
ElevenLabs SFX	云端 API	音效生成

4.6 免费素材库密钥

Pexels、Pixabay、Unsplash 的开发者密钥均可免费申请。这三家属于图库素材（stock media）类别：智能体按语义检索真实拍摄素材而非生成素材。将三个密钥加入 .env 后，真实素材路径的画面来源会显著增加，而成本仍然为零。

4.7 逐步启用与验证策略

配置提供商的关键原则是：**先验证免费与单项能力，再逐个增加付费能力。**具体做法：

1. 建议第一轮只加入三个免费图库密钥，重新运行第 2.6 节注册表命令，确认提供商菜单出现 Pexels、Pixabay、Unsplash。
2. 用一段 15 至 30 秒的测试提示词实际调用一次，确认素材检索生效。
3. 一轮只加一个付费提供商。将全部密钥同时写入会导致 401（认证失败）、429（触发限流）、额度不足、模型不可用等多种错误混在一起，难以定位属于哪个提供商。
4. 每加入一个提供商后重新运行注册表命令，以「能力列表出现新增项」作为生效依据，而不是以「密钥已写入」为依据。

4.8 中文内容的语言选择

制作中文旁白时，应选择支持中文的语音提供商。理由在于：若目标音色不支持中文，会以英文音色硬读中文文本，效果通常不可接受。官方表格中适合中文的选项包括 Google TTS（官方标注 700+ 声音、50+ 语言，适合本地化场景）与 DashScope、豆包、Fish Audio（多语言

与表现力语音选项）。Piper 为本地免费选项，是否包含适合的具体中文音色以其自身发布为准。配置后建议先以一小段中文文本试听，再进入正式生产。

4.9 密钥安全

.env 文件包含可直接调用付费服务的凭据，应做到：

- 不将其提交到 Git。提交前执行 `git status --short` 确认 .env 未进入暂存范围。
- 不在对话中明文粘贴完整密钥；向智能体说明「已配置某提供商」即可，无需提供密钥内容。
- 轮换（作废重建）泄露过的密钥属于标准安全做法。

第 5 章 流水线体系：12 条流水线与七阶段流程

本章解决「选对生产路径」的问题。OpenMontage 把每类视频内容对应到一条预定义流水线，每条流水线内部又分为七个阶段。完成本章学习后，读者应能根据内容目标选择流水线，理解七阶段流程中每个阶段的职责与产物。

5.1 流水线的含义

流水线（pipeline）是「从创意到成品视频」的完整生产工作流。每条流水线都以 YAML 清单定义（现阶段、工具、审查标准、成功关卡），每个阶段由对应的导演技能文件教导智能体如何执行。这种结构意味着：同一条提示词在不同流水线下会走完全不同的生产路径，因此选对流水线是获得理想结果的第一要素。官方将「每个视频请求都视为一个流水线选择问题」写入智能体契约。

5.2 流水线总表

官方仓库标注共 12 条流水线；README 总表列出下述 11 条常规流水线，另有一条角色动画（character-animation）流水线在渲染引擎文档中出现（SVG/GSAP 角色动画，经 HyperFrames 渲染，见第 8 章）。

流水线	产出内容	最适用场景
动画解说（Animated Explainer）	含研究、旁白、视觉、音乐的 AI 生成解说	教育内容、教程、主题解析
动画（Animation）	动态图形、动态排版、动画序列	社交媒体、产品演示、抽象概念
化身代言（Avatar Spokesperson）	数字人驱动的演讲者视频	企业通讯、培训、公告
电影级（Cinematic）	预告片、前导片、基于情绪的剪辑	品牌宣传片、前导预告、促销
片段工厂（Clip Factory）	从单一长素材批量生成并排序的短视频片段	长内容重制为社交媒体短片
纪录片蒙太奇（Documentary Montage）	从 CLIP 索引的免费图库与开放档案剪辑的主题蒙太奇	视频随笔、情绪短片、无付费 API 的真实素材视频
混合（Hybrid）	源素材 + AI 生成的辅助视觉	用图形增强既有画面
本地化与配音（Localization & Dub）	为既有视频添加字幕、配音与翻译	多语言分发
播客重制（Podcast Repurpose）	播客精彩内容转为视频	播客营销、音频可视化
屏幕演示（Screen Demo）	精细打磨的软件录屏与演示	产品演示、教程、文档
口播（Talking Head）	以真人出镜为主的演讲视频	演示文稿、vlog、访谈

选型示例：面向开发者的产品功能讲解应选屏幕演示或动画解说；把两小时播客切成十几条短视频应选片段工厂；把手头口播录像打磨成可发布的演讲视频，应选口播或混合。

5.3 七阶段通用流程

所有流水线遵循同一结构化流程，官方写法为：

```
research -> proposal -> script -> scene_plan -> assets -> edit -> compose
```

对应中文与各阶段产物如下。

阶段	中文	主要工作与产物
research	研究	15–25 次以上网络检索，产出带引用的研究简报
proposal	提案	给出概念方案、工具路径、成本估算，等待人工批准
script	脚本	在研究简报基础上撰写旁白脚本，等待人工批准
scene_plan	场景规划	将脚本拆成逐场景的画面计划（场景卡），等待人工批准
assets	资产生成	生成或检索图像、片段、音频等素材，按场景等待审批
edit	剪辑	确定时间线与剪辑决策（edit_decisions）
compose	合成	由渲染引擎生成最终视频，随后进入渲染后自我审查

5.4 导演技能与流水线清单

阶段导演技能是 skills/pipelines/ 下的 Markdown 指令文件，具体教导智能体在某一阶段应执行什么动作、使用什么工具、达到什么质量标准。流水线清单是 pipeline_defs/ 下的 YAML 文件，声明流水线的阶段构成、可用工具、审查标准与成功关卡。二者的分工是：清单定义「有什么与按什么标准审」，技能定义「具体怎么做」。这两类文件均为可读文本，使用者在第 13 章可以查看乃至修改它们。

5.5 网络研究阶段

研究阶段被官方称为一等公民（first-class stage）：在写下脚本第一句之前，智能体会在 YouTube、Reddit、Hacker News、新闻站点与学术资源上执行 15 至 25 次以上检索，收集数据点、受众问题、热门角度与视觉参考，并将全部内容与引用写入结构化研究简报。此举使视频内容立足于真实、当前的信息，避免事实性幻觉。审批脚本阶段时，核对简报中的引用来源是控制事实质量的关键动作。

5.6 智能体优先架构下的运行方式

OpenMontage 不含代码编排器：AI 编程助手本身就是编排器。使用者的对话是唯一入口——需求、修改、批准全部在对话中完成；自检、状态保存与质量关卡由智能体按契约自动执行。这决定了使用方式：不需要学习专门的界面操作，需要学习的是第 6 章的提示词方法与第 7 章的审批方法。

第 6 章 提示词与创意控制

本章解决「如何让产出接近预期」的问题。OpenMontage 的提示词不是咒语，而是一份微型制作需求书。完成本章学习后，读者应能写出覆盖关键要求的提示词，并掌握参考视频驱动与迭代修改两种高效的创意控制方式。

6.1 提示词的基本要素

对官方全部示例提示词归纳，一条有效的制作需求通常包含以下要素。此归纳为经验性总结，供书写时核对。

要素	作用	示例表述
时长	决定脚本长度、镜头数与成本量级	45 秒、90 秒
流水线意图	将请求映射到对应流水线	animated explainer（动画解说）、documentary montage（纪录片蒙太奇）
主题与受众	决定研究检索方向与文案口吻	面向高中生的 CRISPR 主题
旁白与音乐	声明是否需要及何种风格	无旁白、挽歌式基调、需要音乐
素材约束	决定画面走哪条生成路径	只使用真实素材、使用 AI 生成视觉
成本与工具约束	控制花费与调用范围	只使用免费工具

要素不必每条都写全，但时长与素材约束这两项对结果影响最大，建议始终明确。

6.2 官方示例提示词分级

官方按配置状态将示例分为三档，对应不同的成本量级。

第一档，零密钥需求（成本为零）：

```
"Make a 45-second animated explainer about why the sky is blue"
```

```
"Create a 60-second video about the history of the internet, with narration and captions"
```

```
"Make a data-driven explainer about coffee consumption around the world"
```

第二档，已配置图像/视频提供商（官方标注约 0.15 至 1.50 美元）：

```
"Create a 30-second Ghibli-style animated video of a magical floating library in the clouds at golden hour"
```

```
"Create an animated explainer about how CRISPR gene editing works, using AI-generated visuals"
```

```
"Make a product launch teaser for a fictional smart water bottle called AquaPulse"
```

第三档，完整配置（官方标注约 1 至 3 美元）：

```
"Create a cinematic 30-second trailer for a sci-fi concept: humanity receives a warning from 1000 years in the future"
```

```
"Make a 90-second animated explainer about quantum computing for middle school students, with a fun narrator voice and custom soundtrack"
```

6.3 参考视频驱动的创作

从一条喜欢的视频出发往往比凭空写提示词更高效。OpenMontage 支持从 YouTube 视频、Short、Reel、TikTok 或本地片段出发：粘贴参考视频后，智能体分析其文案、节奏、场景、关键帧与风格，随后产出 2 至 3 个差异化概念、一条诚实的工具路径、成本估算以及正式生产前的小样。官方明确其返回内容包括四项：从参考中保留什么（节奏、开头钩子风格、结构、基调）、改变什么（主题、视觉处理、切入角度、旁白方式）、在目标时长下的预估成本、以当前可用工具实际渲染的效果形态。官方示例如下。

```
"Here's a YouTube short I love. Make me something like this, but about quantum computing."
```

中文含义：给出一条喜欢的 YouTube Short，制作一条类似的、但主题为量子计算的视频。

```
"Analyze this Reel and give me 3 original variants I could make for my own product launch."
```

中文含义：分析这条 Reel，为产品发布给出三种原创变体方案。

参考视频用于约束节奏、结构与镜头语言，而不应复制其脚本或素材；第 4.7 节的逐项验证原则同样适用于参考驱动的首次尝试。

6.4 迭代修改

每一个创意决策点都会暂停等待批准。修改不需要重新开题，直接在对话中说明即可，例如「第 3 场的旁白换成疑问句开头」「把背景音乐换成钢琴、整体放慢」。智能体以检查点（checkpoint）保存状态：状态 JSON 含决策日志与成本快照，支持中断后恢复，因此返工通常只发生在受影响的场景范围，而不会推翻整个项目。

6.5 提示词画廊

仓库根目录的 PROMPT_GALLERY.md 收录了经过测试的提示词，均标注预期成本与输出示例。起步阶段优先选用画廊提示词，确认系统行为后再自写提示词，能显著减少变量。

第 7 章 Backlot 看板与人工审批

本章解决「如何监督与控制制作过程」的问题。对话只告诉使用者智能体说了什么，Backlot 看板则展示生产实际正在做什么。完成本章学习后，读者应能启动看板、理解五个审批门的位置与含义，并掌握按场景审批资产的操作方式。

7.1 Backlot 是什么

Backlot 是一个本地运行的实时制作看板：生产开始后它会自动打开，看板内容完全由流水线已写入的项目文件推导而来，无需额外配置。看板上，阶段会逐一点亮、脚本以剧本页形式落位、场景卡在资产生成时闪烁更新，每一次提供商决策与每一美元支出都展示在墙面上。

7.2 启动与使用

```
python -m backlot open                    # 打开库—本机所有生产项目
python -m backlot open <project-id>       # 打开某一部生产的实时看板
python scripts/backlot_simulate_run.py    # 没有生产在跑时，观看一次模拟运行
```

第三条命令适合在第一次真实制作前熟悉看板元素的语义。运行结束后，看板提供「REPLAY RUN」回放功能，可按时间戳擦洗回放整部生产过程。细节说明见 backlot/README.md。

7.3 审批门机制

人工审批门是强制的，不是建议性的。官方明确以下五个门：提案、脚本、场景规划、生成资产与发布。执行机制为：检查点写入器会拒绝一个未记录人工批准却被标记为「已完成」的关卡阶段；每次修订后被取代（superseded）的检查点会归档保存，使包括门转换在内的完整审计轨迹在修订之后仍可追溯。审批动作发生在对话中；看板会显示当前在等待什么、为什么等待。

7.4 资产总览表审批

资产生成会按场景逐张暂停，呈现一张接触表（contact sheet）：每个镜头的画面（takes）、所用提示词、单项资产成本与质量评分一并列出。这一设计的目的，是让画面在渲染之前而不是渲染之后得到确认——此时拒绝或替换单项资产的成本远低于成片返工。审批方式为在对话中说明：批准哪些场景、哪些场景需要重生成、生成时强调什么。

7.5 从审批到继续

看板指示等待原因，对话完成决策。二者的配合关系可以概括为：看板负责信息透明，对话负责决策输入。审批通过后流水线自动推进到下一阶段，无需额外命令。

第三部分 高级篇

第 8 章 渲染引擎深入：Remotion 与 HyperFrames

本章解决「成片由什么技术栈渲染、如何被选定与锁定」的问题。OpenMontage 的最终合成由可互换的渲染运行时完成，理解引擎差异有助于预判不同提示词的成片质感。完成本章学习后，读者应能读懂提案中的 render_runtime 决策，并知道在何种内容形态下默认使用哪一引擎。

8.1 三种渲染运行时概览

引擎	类型	承担职责
Remotion	本地 Node.js	基于 React 的编程式视频合成
HyperFrames	本地 Node.js（22 及以上）	基于 HTML/CSS/GSAP 的编程式视频合成
FFmpeg	本地	核心视频组装、编码、字幕烧录、音频混音、色彩调色

ffprobe 之外，FFmpeg 层面还内置一组始终免费可用的后期制作工具。

工具	功能
Video Stitch	多片段组装、交叉淡化、画中画、空间布局
Video Trimmer	精确剪切与提取
Audio Mixer	多轨混音、闪避处理（ducking，即人声出现时自动压低背景音乐）、淡入淡出
Audio Enhance	降噪、音量标准化
Color Grade	基于 LUT（颜色查找表）的色彩调色
Subtitle Gen	由时间戳生成 SRT/VTT 字幕文件

8.2 Remotion 详解

Remotion 以 React 组件的方式描述视频，适合数据驱动型内容。官方标注的能力包括：弹簧动画图片场景、数据揭示（stat reveals）、章节标题、主视觉卡片、TikTok 风格逐词字幕、场景过渡（淡入淡出、滑动、擦除、翻转）、Google Fonts、带淡化曲线的音频，以及 TalkingHead 数字人合成。当未配置任何视频生成提供商时，智能体生成静态图像，由 Remotion 将它们转化为完整动画视频——这也是零 Key 路径的成片引擎。Remotion 是数据驱动解说类内容与使用既有 React 场景栈内容时的默认选择。

8.3 HyperFrames 详解

HyperFrames 以 HTML/CSS/GSAP（GreenSock 动画库）表现动态图形，经 `npx hyperframes` 调用，无需拉取整个 monorepo 代码。官方标注的能力包括：动态排版（kinetic typography）、产品宣传与发布短片、自定义动态图形、注册区块（数据图表、噪点覆盖、着色器过渡）、网站转视频工作流，以及绑定 SVG 角色动画。HyperFrames 是以动效为主、用 HTML + GSAP 表达更自然的需求（含角色动画流水线的 SVG/GSAP 输出）时的默认选择。

8.4 render_runtime 治理

渲染运行时在提案阶段选定，并以 render_runtime 字段锁定，其后经 edit_decisions 维持锁定。官方将「在运行时之间静默切换」明确定义为治理违规行为。完整的 Remotion/HyperFrames 决策矩阵见仓库内 skills/core/hyperframes.md。审批提案时，render_runtime 是值得核对的项目之一：它决定了成片的动态风格上限与后续修改的技术路径。

8.5 版本与环境注意

基础安装要求 Node.js 18 及以上，但 HyperFrames 要求 22 及以上。在 Node 18 环境下，若提案选择了 HyperFrames，渲染阶段可能因版本不满足而失败。环境中存在两个 Node 版本时，建议以 nvm 类工具切换至 22 及以上再继续生产。

第 9 章 本地 GPU 与免费生成路径

本章解决「用自有硬件替代付费 API」的问题。对配备 NVIDIA GPU 的使用者，OpenMontage 提供免费的本地视频生成路径；对无 GPU 的使用者，真实素材路径同样是零成本选项。完成本章学习后，读者应能判断本机是否适合启用本地生成，并完成启用、模型选择与基本排错。

9.1 启用本地视频生成

官方提供的启用步骤为两条命令加两项配置：

```
make install-gpu
```

随后在 .env 中加入：

```
VIDEO_GEN_LOCAL_ENABLED=true
VIDEO_GEN_LOCAL_MODEL=wan2.2-ti2v-5b
```

9.2 本地模型选择

官方 README 列出以下可选型号。

模型标识	说明
wan2.2-ti2v-5b	Wan 2.2 文图生视频 5B 版本
wan2.1-1.3b	Wan 2.1 最小规格，资源占用最低
wan2.1-14b	Wan 2.1 大规格，质量更高、要求更强的硬件
hunyuan-1.5	混元视频模型
ltx2-local	LTX-Video 本地版本
cogvideo-5b	CogVideo 5B 版本

对应的提供商侧说明：WAN 2.1/2.2 含免费本地变体与加速的 ComfyUI 工作流；混元免费且质量较高；CogVideo 含 2B 与 5B 两种规格；LTX-Video 本地免费或经 Modal 自托管云端。各模型的具体显存需求不在本仓库文档范围内，应以模型官方发布说明为准；无把握时先从 1.3B 这类小规格起步，再视效果与硬件余量升级。

9.3 本地图像生成与数学动画

无需云端密钥的图像能力同样存在：Local Diffusion 提供免费的 Stable Diffusion 本地生成；ManimCE 在本地执行数学与科学动画渲染。二者均计入工具注册表的能力范围。

9.4 不用 GPU 的免费真实素材路径

无 GPU 或不想下载模型时，纪录片蒙太奇流水线是零成本的「真实视频」路径：系统从 Archive.org、NASA、Wikimedia Commons（以及可选的 Pexels、Unsplash）建立 CLIP 索引语料库。CLIP 是图文语义匹配模型，用于将素材按语义而非关键词建立可检索索引。该路径产出真实拍摄影像的成片，全程无需视频生成 API。触发方式见第 3.4 节。

9.5 排错要点

本地生成失败时按顺序检查三项：CUDA 驱动与环境、显存余量、模型文件是否完整下载。一条来自社区实践的重要判读：`python -c` 能导入模块只说明 Python 层正常，不等于 GPU 推理可用；将二者混为谈到会导致排错方向错误。验证本地推理是否可用的最短路径，是跑一个时长最短的单镜头生成任务，而不是直接上完整项目。恢复配置时遵循第 4.7 节的单一变量原则。

第 10 章 风格系统与平台输出规格

本章解决「成片视觉语言与分发规格」的问题。风格剧本定义成片的视觉语言，平台输出配置定义成片的分辨率与宽高比。完成本章学习后，读者应能知道内置的视觉语言有哪几种、在提示词中如何表达目标平台，以及自定义风格的入口在哪里。

10.1 风格剧本机制

风格剧本（style playbook）是 styles/ 目录下的 YAML 文件，控制排版、配色、运动风格、音频配置与质量规则。智能体读取剧本后，会在全部生成资产上保持一致地应用该视觉语言。这意味着，「画面风格不统一」在 OpenMontage 中是一个由剧本管理的系统性问题，而不是逐镜头手工对齐的问题。

10.2 内置剧本

剧本	英文	最适用场景
干净专业	Clean Professional	企业、教育、SaaS
扁平动态图形	Flat Motion Graphics	社交媒体、TikTok、初创公司
极简图解	Minimalist Diagram	技术深度解析、架构讲解

10.3 平台输出配置

系统为各主流平台内置了渲染规格。

配置	分辨率	宽高比
YouTube 宽屏（Landscape）	1920x1080	16:9
YouTube 4K	3840x2160	16:9
YouTube Shorts	1080x1920	9:16
Instagram Reels	1080x1920	9:16
Instagram 动态（Feed）	1080x1080	1:1
TikTok	1080x1920	9:16
LinkedIn	1920x1080	16:9
电影级（Cinematic）	2560x1080	21:9

10.4 运用方式

在提示词中直接写明目标平台与竖屏/横屏/方形形态（例如「vertical, for TikTok」），提案阶段即会按对应规格确定渲染配置。确定风格有两种途径：在提示词中描述期望的视觉语言，由智能体映射到最接近的内置剧本；或直接修改 styles/ 下的 YAML（自定义方法见第 13 章）。两种途径均建议在脚本审批前完成，避免资产阶段返工。

第 11 章 预算治理与质量关卡机制

本章解决「为什么这个系统很少产出失控的失败件、也不会产生意外账单」的问题。OpenMontage 将视频制作当作真正的工程来治理：质量关卡、审计轨迹与执行控制贯穿各阶段。完成本章学习后，读者应能理解四类自动质量关卡、提供商的七维评分机制、决策审计与预算控制三类治理部件，并据此定位质量问题。

与第 7 章的人工审批门不同，本章所有机制均由系统自动执行，无需人工参与。

11.1 四类质量检验门

关卡	触发时机	机制
合成前验证	渲染启动前	检查交付承诺是否被违反（例如宣称「以运动为主」的视频却有 80% 静态图像）、幻灯片风险评分是否达到危急、渲染器族是否缺失；任一不满足即阻止渲染，在浪费 GPU 时间之前拦截坏计划
渲染后自我审查	每次渲染完成后	运行 ffprobe 验证，在 4 个位置抽帧检查黑屏与破损覆盖层，分析音频电平是否存在静音与削峰，验证交付承诺是否兑现，并检查字幕存在性；审查不通过则不向使用者呈现该视频
幻灯片风险评分	方案评估阶段	从重复性、装饰性视觉、运动幅度弱、镜头意图、过度依赖排版、不支持的电影级宣称六个维度打分，防止产出「带动画的 PPT」
源文件检查	使用者提供自有素材时	探查每个文件的分辨率、编解码器、音频通道与时长，并在任何创意决策之前形成规划依据；不允许智能体凭文件名臆测内容

其中「交付承诺」（delivery promise）指提案中对该片形态的明确约定，例如时长、画面主体性质、是否含旁白与字幕；它同时是合成前验证与渲染后自我审查的核对依据。

11.2 七维评分的提供商选择

每一次工具选型（视频生成、图像生成、TTS、音乐）都经过同一评分引擎；官方给出的维度与权重如下。

维度	权重
任务契合度	30%
输出质量	20%
控制功能	15%
可靠性	15%
成本效益	10%
延迟	5%
连续性	5%

选择器在打分前会对松散的需求上下文做标准化扩充（例如将「皮克斯风格、需保持角色一致」扩展为评分器可用的意图与风格信号），并在输出中附带被选中提供商对应的智能体技能名称，供智能体在编写提示词前先阅读相应的第三层知识。获胜提供商及其得分、以及所有被考虑的备选方案，一并记入决策轨迹。

11.3 决策审计轨迹

每一个重大创意与技术选择——提供商、风格剧本、音乐曲目、音色、渲染器族系、任何后备或降级——都会记录：考虑过的备选项、置信度得分与选择理由。累计决策日志跨全部阶段持久保存，因此任何一处「为什么长这样」都可以回溯。审批与定位问题时，它是核对智能体行为的原始依据。

11.4 预算控制

预算治理遵循「预估—锁定—结算」三步：执行前展示预计成本；施工前锁定预算；结算时记录实际开销。控制模式共三种：observe（仅跟踪）、warn（超支记录警告）、cap（硬性上限）。另有两项阈值：按动作审批阈值，超过即暂停等待确认，默认 0.50 美元；总预算上限，默认 10 美元，二者均可配置。官方表述为「没有意外账单」——智能体在花费之前会告知将要花费多少。

11.5 治理与使用策略

对日常使用者，治理机制转化为三条操作准则：一是任何「已完成」的说法都应以可播放成片通过自我审查与 ffprobe 验收为准；二是质量问题的定位从决策日志入手，而非从画面倒推；三是控制成本的杠杆在提案门与资产接触表，而不是渲染之后。

第四部分 精通篇

第 12 章 架构解密：目录、注册表与三层知识体系

本章解决「系统如何组织知识与状态」的问题。OpenMontage 的设计特点是「Python 提供工具与持久化，全部编排逻辑与质量标准以可读文本存在」。完成本章学习后，读者应能读懂目录结构与执行流程，理解三层知识架构，并知道状态与决策记录存放在哪里。

12.1 目录结构

以英文 README 当前口径（英文与中文 README 的规模数字略有出入，本教程以英文 README 为准）：

```
OpenMontage/
├── tools/              # 100+ 注册生产工具（智能体的“手”）
│   ├── video/          # 20+ 视频生成提供商 + 合成(compose)、拼接(stitch)、裁剪(trim)
│   ├── audio/          # 10+ 语音提供商 + 音乐、混音、增强
│   ├── graphics/       # 15+ 图像提供商 + 图表、代码片段、数学动画
│   ├── enhancement/    # 放大、移除背景、面部增强、色彩调色
│   ├── analysis/       # 转写、场景检测、帧采样
│   ├── avatar/         # 数字人（talking head）、唇形同步
│   └── subtitle/       # SRT/VTT 字幕生成
│
├── pipeline_defs/      # YAML 流水线清单（智能体的“剧本”）
├── skills/             # Markdown 技能文件（智能体的“知识”）
│   ├── pipelines/      # 各流水线的阶段导演技能
│   ├── creative/       # 创意技法技能
│   ├── core/           # 核心工具技能
│   └── meta/           # 审阅者、检查点协议
│
├── schemas/            # 20+ JSON Schema（契约验证）
├── styles/             # 视觉风格剧本（YAML）
├── remotion-composer/  # React/Remotion 视频合成引擎
├── lib/                # 核心基础设施（配置、检查点、流水线加载器）
└── tests/              # 契约测试、QA 集成测试、评估套件
```

增强与分析工具与第 8 章的后期制作工具同属一条能力线：放大（Upscale，基于 Real-ESRGAN）、移除背景（rembg/U2Net）、面部增强、面部修复（CodeFormer/GFPGAN）、转写（WhisperX，含词级时间戳）、场景检测、智能帧采样、视频理解（CLIP/BLIP-2 视觉语言分析）；数字人侧有 SadTalker/MuseTalk 数字人动画、Wav2Lip 唇形同步与 Kling 官方云端数字人与唇形同步。

12.2 智能体优先的执行流程

系统不设代码编排器，一次完整生产的执行序列如下（自上而下）。

1. 使用者在对话中提出需求；
2. 智能体读取流水线清单（YAML），获取阶段、工具、审查标准与成功关卡；
3. 智能体读取各阶段导演技能（Markdown），获知每个阶段的具体做法；
4. 智能体调用 Python 工具，评分选择器在七个维度上为每次选型排序；
5. 智能体以审阅者技能自我审查：Schema 验证、剧本依从性检查、质量检查；
6. 智能体保存状态检查点（JSON），含决策日志与成本快照，支持恢复；
7. 智能体在每个创意决策点提交人工批准；
8. 通过合成前验证关卡（交付承诺、幻灯片风险、渲染器治理）；
9. 渲染（Remotion 或 FFmpeg），合成引擎与视觉语法匹配；
10. 渲染后自我审查（ffprobe、抽帧、音频分析、承诺验证）；
11. 仅当自我审查通过，才输出最终视频。

全部创意决策、编排逻辑、审查标准与质量标准都存在于可读文件（YAML 清单 + Markdown 技能）中，使用者可以检查与定制——这是第 13 章扩展工作的基础。

12.3 三层知识架构

系统将知识分为三层，每一层回答一个问题。

```
第 1 层: tools/ + pipeline_defs/    “有什么”   — 可执行能力 + 编排
第 2 层: skills/                   “如何使用” — OpenMontage 的约定与质量门槛
第 3 层: .agents/skills/           “原理为何” — 外部技术知识包
```

每个工具声明它依赖哪些第 3 层技能。智能体读第 1 层了解可用能力，读第 2 层了解 OpenMontage 期望的使用方式，需要深入时读第 3 层获取某个外部技术的知识包。调试与扩展时应遵循同一顺序：先确认能力存在（第 1 层），再核对约定（第 2 层），必要时才深入原理（第 3 层）。

12.4 工具注册表与契约校验

工具注册表自动发现并注册 tools/ 下符合契约的工具——新增工具无需手动注册，这由「继承 BaseTool 并实现工具契约」保证（见第 13 章）。第 2.6 节两条命令中的 support_envelope 与 provider_menu 即注册表的两个查询接口。阶段产物则受 schemas/ 下 20 多个 JSON Schema 约束，由审阅者技能执行 schema 验证，使阶段间数据传递具备契约保障。

12.5 检查点与可恢复性

每个阶段结束时，系统在项目内保存状态检查点 JSON，内容包括决策日志与成本快照。检查点有两个用途：中断与失败后从断点恢复，而不是重跑整个流水线；修订后保留全部历史——被取代的检查点会被归档，使审批轨迹和门转换记录不随修订丢失。查错时，检查点与决策日志是定位「系统做了什么、为什么」的第一入口。

第 13 章 定制与扩展：添加工具、流水线与风格

本章解决「把系统改成自己的生产线」的问题。OpenMontage 的设计目标之一是可扩展：注册表自动发现新工具，清单驱动一切编排。完成本章学习后，读者应能按照官方步骤添加自定义工具与流水线、修改风格剧本，并以契约测试验证改动的正确性。

13.1 添加新工具

官方给出的四步：

1. 在 tools/ 的对应子目录中创建一个 Python 文件；
2. 继承 BaseTool 并实现工具契约；
3. 注册表会自动发现该工具——无需手动注册；
4. 若该工具需要使用指导，为它添加一个对应的技能（skill）文件。

工具契约的含义在仓库内以 BaseTool 基类与现有工具的实现为准；动手前建议先阅读 tools/ 下同类工具的既有实现，保持接口风格一致。

13.2 添加新流水线

官方给出的三步：

1. 在 pipeline_defs/ 中创建一个 YAML 清单；
2. 在 skills/pipelines/<新流水线名称>/ 下创建各阶段导演技能文件；
3. 引用现有工具——或在确有需要时新增工具。

新清单中的阶段、审查标准与成功关卡写法，应参照既有流水线的 YAML 文件；最简方式是在与目标形态最接近的既有流水线上复制修改。

13.3 修改与自定义风格剧本

styles/ 下的 YAML 剧本控制排版、配色、运动、音频配置与质量规则。自定义风格剧本的最短路径是以某个内置剧本为模板复制修改，命名为新剧本后即可在提示词或提案中被选用。

13.4 调整审查与质量标准

由于全部编排逻辑、审查标准与质量标准都存放于可读文件（YAML 清单 + Markdown 技能），对验收尺度与流程的定制同样通过修改这些文件完成。应当注意：修改审查标准会同时改变智能体的自我审查行为与关卡放行条件，改动应保守、可回退并留下说明。

13.5 测试与验证

仓库内置两类测试命令：

```
# 运行契约测试（无需任何 API 密钥）
make test-contracts

# 运行全部测试
make test
```

改动工具或清单后，先跑 make test-contracts 再跑完整测试；契约测试不依赖密钥，适合作为提交前的快速回归手段。

13.6 改动安全与社区渠道

动手修改前保留现场：确认工作区干净（`git status --short`），必要时先提交或暂存当前改动，使自定义内容始终独立于官方更新。遇到上游更新导致改动冲突时，将自有修改集中为少量文件的做法维护成本最低。

社区渠道：GitHub Discussions 设有 Show and Tell（展示作品与提示词）、Ideas（提议新流水线、工具、风格与集成）与 Q&A（安装、流水线与排查问题）三个板块；官方 YouTube 频道每条视频附完整提示词、流水线、工具与成本，用于复现。深入文档：docs/ARCHITECTURE.md（完整技术参考）、docs/PROVIDERS.md（提供商设置、定价与免费额度）、AGENT_GUIDE.md（智能体契约）、PROJECT_CONTEXT.md（架构参考）。

第 14 章 综合实战：从需求到成片的完整案例

本章将前十三章的知识串联为一次完整制作：面向开发者的 60 秒动画解说，介绍一款虚构的日志监控工具，目标平台为 YouTube 宽屏，预算上限 3 美元。完成本章学习后，读者应能独立完成同等复杂度的制作项目，并建立可复用的生产流程。

14.1 案例设定与前置检查

假设第 1 至 3 章的安装与零 Key 验收已完成：注册表两条命令正常，一条 15 至 30 秒测试短片已通过 ffprobe 验收（附录 B）。本案例使用 OPENAI_API_KEY 单一密钥配置（TTS + GPT 图像），并已申请 Pexels 免费密钥——这是官方示例 VOID — Neural Interface（总成本 0.69 美元）所验证过的最小付费配置。

14.2 第一步：提出需求

在 AI 编程助手中打开仓库，发出需求。提示词覆盖第 6.1 节全部要素：

```
"Make a 60-second animated explainer about a fictional log-monitoring tool called LogSift,
for a developer audience. Clean professional style, with a clear hook in the opening,
using AI-generated visuals, narration and word-level captions. Estimate cost before any
asset generation and keep it under $3."
```

中文含义：为一款名为 LogSift 的虚构日志监控工具制作 60 秒动画解说，面向开发者受众；干净专业风格，开头有清晰钩子；使用 AI 生成画面、旁白与词级字幕；在任何资产生成之前先估算成本，总支出控制在 3 美元以内。

14.3 第二步：提案审批

提案门暂停时核对五项：流水线选择（应为动画解说）；render_runtime（数据驱动解说默认 Remotion，本案例应出现 Remotion 字样）；成本估算是否在预算内并低于按动作审批阈值；交付承诺的表述（时长 60 秒、AI 生成画面、旁白、词级字幕）；提供商选择及其评分理由（决策日志应展示被考虑的备选）。任何一项不合理，在对话中说明并要求重提，而不是带着隐患放行。

14.4 第三步：研究简报、脚本与场景规划审批

研究简报应包含对「日志监控工具」品类的事实、竞品表述与开发者关注点，每条数据带来源引用；核对来源是本阶段的核心动作。脚本审批时检查：60 秒时长对应的旁白字数是否合理、开头钩子是否成立、技术表述与研究简报一致。场景规划审批检查：每个场景卡的画面描述、预计展示方式与画面来源类型（AI 生成或图库）；此时应同步确认风格剧本（干净专业）与平台规格（1920x1080，16:9）已体现在计划中。

14.5 第四步：资产接触表审批

资产生成按场景暂停。接触表列出每个镜头的画面、生成提示词、单项成本与质量评分。逐场景决策：批准合格镜头；对不合格镜头在对话中说明问题（例如「监控界面应显示错误日志而非文本文件」「配色偏离品牌主色」）并要求重生成。此阶段允许反复，单项资产成本低于整片成本，返工代价最小。

14.6 第五步：渲染与人工复核

批准全部资产后，流水线进入剪辑与合成。系统自动执行合成前验证与渲染后自我审查；自我审查不通过时，看板会显示原因，智能体应自行修复后重新渲染。系统放行后仍应做人工复核：定位 MP4，运行第 3.5 节的 ffprobe 命令，确认时长、视频流与音频流符合提案承诺，并抽样检查字幕与旁白同步。

14.7 第六步：迭代与再生产

入库后最常见的三类迭代由检查点机制支撑：改时长（延长或压缩脚本后仅影响受影响场景）、换画风（重跑资产阶段并选定新风格剧本）、多语言分发（改选本地化与配音流水线，复用同一时间线结构）。每次成片完成应归档三件套：最终提示词、提案成文（含 render_runtime 与成本）与验收记录，构成可复现的生产基线。

14.8 从单次实验到稳定生产

将本案例流程固化为稳定生产的三条经验：其一，先零 Key 或最小付费配置建立可重复基线，再逐项增加提供商，任何回归都用同一测试提示词复验；其二，把审批门当作质量杠杆——提案控制成本与形态，脚本与简报控制事实，接触表控制画面，渲染后审查只做最后的机械确

认；其三，自定义内容（提示词模板、风格剧本、验收清单）与上游仓库分离存放，使官方更新与自有定制互不干扰。至此，本教程从安装、首片、进阶配置、深度定制到完整生产的全部内容即告完成。

附录

附录 A 故障排查手册

下表汇总本教程涉及环境的常见故障现象、应优先检查的项目与判断标准。判断标准一栏中的命令均可直接执行。

现象	先检查	判断标准
ModuleNotFoundError	当前 Python 解释器路径与虚拟环境	`python -c "import sys; print(sys.executable)"` 的输出应指向 .venv
ffmpeg 命令找不到	PATH 与 FFmpeg 安装	重新打开终端后 `ffmpeg -version` 仍应成功
npm install 报 ERR_INVALID_ARG_TYPE	Node/npm 版本	按官方说明改用 `npx --yes npm install`，且安装应零退出
PowerShell 拒绝激活脚本	执行策略设置	确认策略问题而非环境损坏；或改用 `.\.venv\Scripts\python.exe`
Provider 请求失败	对应环境变量、账户额度与提供商日志	区分 401（认证）、429（限流）、超时与模型不存在；不要连续更换多个密钥
有画面无声音	TTS 产物、音轨与 ffprobe 输出	成品 MP4 中应存在预期的音频流
智能体声称完成但没有成片	项目输出目录与最终质检步骤	必须找到可播放的 MP4 并通过 ffprobe；只有项目文件不算完成
本地 GPU 生成失败	CUDA 环境、显存余量、模型下载完整性	逐项排查；先跑最短的单镜头任务验证推理可用性
渲染阶段失败（HyperFrames）	Node.js 版本	HyperFrames 要求 Node.js 22 及以上
安装状态损坏	代码是否被智能体改动	`git status --short` 与 `git rev-parse --short HEAD` 确认状态

恢复到可工作安装状态的做法：不要直接覆盖整个仓库。先按上表末行确认代码是否被改动，再将旧虚拟环境改名保留（例如改名为 .venv-broken），随后按第 2.4 或 2.5 节步骤重建新的 .venv；恢复的验收标准不是「报错消失」，而是同一个测试短片重新生成并通过 ffprobe 检查。逐步定位遵循单一变量原则：一次只更换 Python、Node、FFmpeg 或提供商中的一个环节。

附录 B 安装与出片验收清单

逐项核对，全部通过才视为「可稳定生产」状态。

1. Python、Node.js、FFmpeg 在新打开的终端中都能正常输出版本；
2. 第 2.6 节的两条注册表命令零退出，并显示符合预期的能力与提供商；
3. 一个 15 至 30 秒的测试任务生成了可播放的 MP4；
4. ffprobe 显示该 MP4 具有非零时长以及预期的视频流、音频流；
5. .env 与生成的素材文件未进入 Git 暂存范围（以 `git status --short` 核对）；
6. 已实际演练回退：移除最近新增的某一提供商后，系统回到零 Key 或上一可用配置。

附录 C 术语表

中文术语	英文	简要定义
智能体	Agent	运行 OpenMontage 的 AI 编程助手，承担编排与执行双重角色
智能体优先架构	Agent-first architecture	不设代码编排器、由智能体按契约文件编排一切的系统架构
流水线	Pipeline	从创意到成品的完整生产工作流，共 12 条
流水线清单	Pipeline manifest	pipeline_defs/ 下的 YAML 文件，定义阶段、工具、审查标准与成功关卡
阶段	Stage	流水线的一环；通用流程为研究、提案、脚本、场景规划、资产生成、剪辑、合成七阶段
导演技能	Director skill	教导智能体执行某阶段的 Markdown 指令文件
工具注册表	Tool registry	自动发现并注册工具的机制，提供能力范围与提供商菜单查询
能力范围	Support envelope	当前机器经注册表确认可支持的生产能力集合
提供商	Provider	执行具体生成的服务或模型（云端 API、本地模型、图库）
评分选择器	Scored selector	以七维加权评分自动选择提供商并记录决策的引擎
质量检验门	Quality gate	系统自动执行的质检机制；本教程列为四类，见第 11 章
审批门	Approval gate	需人工批准方可继续的创意决策点，共五处
检查点	Checkpoint	阶段性保存的状态 JSON，含决策日志与成本快照，支持中断恢复
决策日志	Decision log	记录每次重大选择的备选项、置信度与理由的审计文件
交付承诺	Delivery promise	提案中关于成片形态的明确约定，是合成交验的核对依据
幻灯片风险评分	Slideshow risk scoring	六维度评估「带动画的 PPT」倾向的机制
源文件检查	Source media inspection	对使用者自有素材逐文件探查并形成规划依据的机制
Backlot 看板	Backlot	本地实时生产看板，由项目文件自动生成
渲染运行时	render_runtime	提案阶段选定并锁定的合成引擎（Remotion 或 HyperFrames）
Remotion	Remotion	基于 React 的编程式视频合成引擎
HyperFrames	HyperFrames	基于 HTML/CSS/GSAP 的编程式视频合成引擎，要求 Node.js 22+
GSAP	GSAP	GreenSock 动画库，HyperFrames 的动效基础
FFmpeg	FFmpeg	视频组装、编码、字幕烧录、音频混音与调色的本地工具
ffprobe	ffprobe	FFmpeg 附带的媒体信息探测工具，用于验收输出
词级字幕	Word-level captions	按单词级时间戳逐词出现的字幕
风格剧本	Style playbook	styles/ 下的 YAML 文件，统一控制成片的视觉语言
平台输出配置	Platform output profile	针对各平台的分辨率与宽高比预置规格
CLIP	CLIP	图文语义匹配模型，用于素材语料库的语义检索
语料库	Corpus	从开放档案与图库建立的 CLIP 索引素材集合
数字人	Talking head	由照片或素材驱动的口播形象能力
唇形同步	Lip sync	以音频驱动嘴型的对齐处理
闪避处理	Ducking	人声出现时自动压低背景音乐的混音手法
LUT	LUT	颜色查找表，用于统一调色
PPT（幻灯片）风险	Slideshow risk	见「幻灯片风险评分」
零 Key 路径	Zero-key path	不配置任何付费密钥即可完成制作的路径

> 本教程基于 2026 年 10 月 22 日核实的官方公开资料编写。OpenMontage 迭代较快，执行本教程中的命令前，建议先核对仓库当前 README；各提供商定价与免费额度以 docs/PROVIDERS.md 为准。